

Roteiro — Entrega 1

Rate-Based ABR · Cliente funcional · Demo ao vivo

5 minutos | FALA = dizer AÇÃO = fazer na tela ATENÇÃO = cuidado

1. Abertura

0:00 – 0:40

Contexto rápido

Antes de começar já ter aberto:

Terminal na pasta `client/` | `abr.py` no editor (linha 18) | Wireshark capturando em `wlp1s0`

“Vamos mostrar o cliente de streaming adaptativo da Entrega 1 rodando contra o servidor real. O cliente baixa o manifest, mede a vazão de cada segmento, escolhe a qualidade pelo algoritmo Rate-Based ABR e grava tudo em CSV com gráficos automáticos.”

2. Código — ABR e Buffer (rápido)

0:40 – 1:30

Mostrar o código

Ir para o editor → `abr.py` linha 52 — destacar a linha `safety_limit = throughput_kbps * self.SAFETY_FACTOR`

Depois ir para `buffer_manager.py` linha 45 (`add_segment`) e linha 65 (`consume`)

“O ABR calcula `safety_limit = vazão × 0,85` e escolhe a maior qualidade cujo bitrate caiba nesse limite — os 15% de margem evitam rebuffering em flutuações de rede.

O `BufferManager` faz `buffer += 2s` a cada segmento recebido e `buffer -= tempo_de_download` pelo tempo que o player ficou esperando. Se zerar, rebuffering. Se $\geq 2s$, o campo `buffer_can_play` é 1.”

3. Demo ao vivo — cliente + Wireshark

1:30 – 4:00

Demo principal

Wireshark já deve estar rodando. Aplicar o filtro agora se ainda não aplicou:
`ip.addr == 137.131.178.229`

Rodar o cliente no terminal:

```
python3 main.py
```

Aguardar o seg 001 aparecer e começar a falar.

“O manifest foi baixado — no Wireshark viram o SYN e o GET /manifest.

[FAZER: Apontar para a linha seg 001 no terminal]

Segmento 001 saiu em 240p, cold start sem histórico ainda. A partir do 002 o ABR já tem uma medição e sobe a qualidade. Olhem as colunas: **vazao** é a banda medida, **buf** é o buffer crescendo em segundos — ele sobe porque baixamos mais rápido do que a taxa de reprodução — e **dl** é o tempo real de cada download.

[FAZER: Alternar para o Wireshark]

No Wireshark cada rajada de pacotes TCP é um segmento. O intervalo entre rajadas é o processamento do cliente. Quando o jitter no terminal sobe, dá para ver o espaçamento irregular entre os pacotes aqui.

[FAZER: Voltar para o terminal e deixar terminar]

A qualidade estabiliza em 480p porque com o servidor em 2 Mbps e o safety factor, a vazão média de 1.100 kbps fica abaixo do limiar de 1.500 kbps da 720p. Deficiência do Rate-Based puro que vamos resolver na Entrega 2.”

4. CSV e gráfico

4:00 – 4:45

Evidências

```
cat ../logs/$(ls -t ../logs/ | head -1) | column -t -s, | head -8  
xdg-open ../graphs/throughput_timeline.png &
```

“CSV com as 14 colunas da especificação — **buffer_can_play** em 1 em todos os segmentos, sem nenhum rebuffering. O gráfico mostra a vazão e a qualidade selecionada: correlação direta, e o buffer subindo continuamente. Esse é o baseline da Entrega 1.”

5. Encerramento

4:45 – 5:00

Fechar

“Pronto. Cliente funcional, CSV gerado, gráficos automáticos. Alguma pergunta?”

Respostas rápidas

Pergunta	Resposta
Onde o ABR decide?	<code>abr.py</code> linha 52 — <code>safety_limit = throughput * 0.85</code> , itera qualidades ordenadas por bitrate.
Como o buffer é calculado?	<code>buffer += 2s</code> (segmento recebido) — <code>download_time_s</code> (tempo de espera).
O que é <code>buffer_can_play</code> ?	1 se <code>buffer ≥ 2s</code> . Indica reprodução contínua sem interrupção.
Por que parou em 480p?	Banda $2 \text{ Mbps} \times 0,85 \approx 935 \text{ kbps}$ efetivos $< 1.500 \text{ kbps}$ da 720p.
O que é jitter EWMA?	Média móvel exponencial ($\alpha=0,2$) da variação entre segmentos consecutivos.

Checklist pré-apresentação: `curl http://137.131.178.229:8080/health` · Wireshark em `wlp1s0` · Teste rápido `python3 main.py` · Gráficos de backup em `graphs/`